

# LARAVEL PART 1

IT9a/L



# What is Laravel?

Laravel is elegant, expressive and flexible **PHP framework** with extreme focus on clean code and speed which describes itself as “The PHP framework for web artisans”.

It is a free and Open-source PHP framework created by Taylor Otwell, based on **Model View Controller (MVC)** architecture.

Laravel framework provides various PHP libraries and helper functions and can help you to focus on more important pieces while providing common functions and logic to speed up the development time and ease up the development.



# Laravel Installation

Three things you need for your environment:

1. **PHP Environment:** PHP is the core server-side scripting language that powers the Laravel framework.
2. **Composer Dependency Manager:** Composer is an essential dependency management tool for PHP.
3. **The Laravel Framework:** This is the robust, elegant PHP web application framework we will be installing to build scalable, secure, and maintainable web applications.



# Laravel Installation

- Check the PHP version of your unit with the ***php -version*** command in the command prompt.
- Download and install the composer.
- Check the composer version using the ***composer -version*** command in the command prompt.
- Install Laravel installer in the command prompt using this command: ***composer global require laravel/installer***
- Create Laravel project using one of the following commands:

❑ ***laravel new appName***

❑ ***composer create-project laravel/laravel appName***



# Routing 101



# What is Laravel Routing?

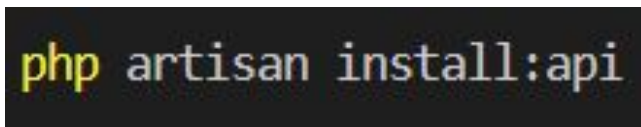
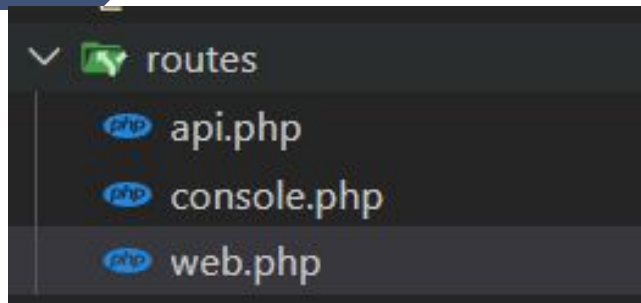
**Laravel's routing system** is the entry point for all HTTP requests to your application. It acts as a traffic director, taking the URL requested by the user and pointing it to the correct logic (like a Controller or a Closure) to handle that request and return a response.

Every request passes through the routing system before it reaches a controller, a view, or even an API response.



# Defining Routes

Defining a route means telling your application, *"When a user makes a specific type of request (GET, POST, etc.) to this specific URL, execute this piece of code."* Most routes are defined in the ***routes/web.php*** file (for web interfaces) or ***routes/api.php*** (for APIs).





# Defining Routes

**Route::get(\$uri, \$callback);**

- **Route:** is a Facade. A facade is a design pattern used by Laravel to provide a simple, memorable syntax to access complex underlying classes.
- **::** : It is a **Scope Resolution Operator** sometimes called the *Paamayim Nekudotayim*. It is used to call static methods on a class.





# Defining Routes

**Route::get(\$uri, \$callback);**

- **get()** : This tells Laravel which HTTP verb (or method) this route should respond to. The **get** method is used for retrieving data—like loading a webpage or fetching a list of items.
- **\$uri** : **URI** stands for ***Uniform Resource Identifier***. In this context, it is a string representing the URL path that the user is visiting, relative to your domain.



# Defining Routes

## **Route::get(\$uri, \$callback);**

- **\$callback** : This tells Laravel what code to execute when an incoming request matches both the HTTP method (get) and the path (\$uri). The callback can take two common forms:
  - ❑ **A Closure (Anonymous Function):** A small block of code written directly in the route file to return a simple response.
  - ❑ **A Controller Array:** A pointer to a specific method within a Controller class, which keeps your code organized when dealing with complex logic.



# Layout Files



# Laravel Layout Files

Laravel layout files are a core feature of **Blade**, Laravel's built-in templating engine. They allow you to define a single master structure for your web pages—such as your `<html>` tags, headers, navigation bars, and footers—and reuse that structure across multiple pages. Instead of copying and pasting the same HTML on every single page, you create one "Master Layout" and have your individual pages (Child Views) inject their unique content into specific placeholders within that master file.



# Laravel Layout Files

- Create **Components** folder.
- Create **layout.blade.php**.
- Use the **{{ \$slot }}** inside the **layout.blade.php**.
- **{{ }}** : In Laravel, the **{{ }}** syntax is a core feature of Blade, Laravel's built-in templating engine. It is used to **display** or **"echo"** data onto a web page.
- **{{ \$slot }}** : acts as the default placeholder inside a component. It dictates exactly where the injected content should appear.
- Use the **<x-layout> </x-layout>** elements in home, about, and contact pages
- put the nav into layout and call the layout to the about, welcome, and contact



# Laravel Layout Files

- Use the **<x-layout> </x-layout>** elements in home, about, and contact pages
- **<x-** : This is a special marker in Laravel Blade. When Laravel sees **<x-**, it knows you are not writing standard HTML (like a **<div>** or **<p>**); instead, you are calling a custom Blade Component.
- **layout** : it searches for a file named **layout.blade.php** inside your **resources/views/components** folder.
- **HTML-Like Structure**: It uses an opening **<x-layout>** tag and a closing **</x-layout>** tag.



# Laravel Layout Files

- add internal css using style tag in layout component : style the nav>a tags
- The title for each page should be dynamic. So use the **{{ \$title }}** then add title attribute and its value to the about, welcome, and contact.
- **{{ \$title }}** : It simply prints the value of a custom variable named "title".
- Use the **title** attribute to other pages and give its value.
- use the **@props([ 'title' => 'Default value of title'])** to the layout page to have a default value.



# Laravel Layout Files

- create **card** component
- add style: **`<div style="background: #e3e3e3; padding: 1rem; margin: auto;"> {{ $slot }} </div>`**
- make the contract use the **x-card** component in p element
- transfer the **.card** style to the layout style element
- create class for the x-card **class="max-w-400"**
- create style using the class in layout with properties **`.max-w-400{max-width: 400px; margin:auto;}`**





# Laravel Layout Files

- In the card component, use the `{{ $attributes->merge(['class' => 'card']) }}` to get the style of `.card`
- `$attributes->merge(['class' => 'card'])` : The `$attributes` is a special variable that automatically catches any extra attributes (like class, id, or data-\* tags) that you pass into your `<x-card>` tag. The `merge()` method acts as a smart mixer. It takes your default, baseline classes (`['class' => 'card']`) and flawlessly combines them with the newly passed classes (`max-w-400`).